

设计报告

目录

- 1. 项目概述与目标
- 2. 模型架构
- 3. 量化方法详解
 - 3.1 STE (直通估计器)
 - 3.2 LSQ+ (可学习步长量化增强版)
 - 3.3 DSQ (可微量化)
 - 3.4 三种方法对比
- 4. 训练流程
- 5. 部署到光子推理

1. 项目概述与目标

本项目面向光子计算阵列实现 MNIST 手写数字分类，核心目标是在保持高准确率的前提下，将网络权重与激活值压缩至 4-bit 整型，从而适配光子 MAC（乘累加）阵列的硬件约束。

光子计算阵列的典型特点包括：

- 只能执行 MAC 运算，不支持加法偏置 (bias=False)；
- 输入/权重通常以低精度整型表示（本项目采用 INT4）；
- 8x2 阵列分片 (tiling) 要求数据按固定维度切分。

为了弥补低精度量化带来的精度损失，本项目采用**量化感知训练 (Quantization-Aware Training, QAT)**，在 PyTorch 训练过程中引入伪量化 (fake quantization)，使模型在有限精度下仍能达到与全精度模型接近的 Top-1 准确率。

2. 模型架构

所有变体共享同一 3 层 MLP 架构，**全程 bias=False**（光子硬件不支持加法偏置）：

Input (784) -> FC1 (784, 128) -> ReLU -> FC2 (128, 64) -> ReLU -> FC3 (64, 10)
-> Output

变体	类名	隐藏层维度	量化方法
全精度基线	PhotonicMLP	128, 64	无 (FP32)
STE 基线	PhotonicMLP_STEQ	128, 64	STE 伪量化 (静态 scale)
LSQ+	PhotonicMLP_LSQPlus	128, 64	LSQ+ (可学习 scale + zero-point)
DSQ	PhotonicMLP_DSQ	128, 64	DSQ (tanh 软量化 + 温度退火)

模型定义位于：

- `src/models/base.py` — PhotonicMLP
- `src/models/qat_steq.py` — PhotonicMLP_STEQ
- `src/models/qat_lsplus.py` — PhotonicMLP_LSQPlus
- `src/models/qat_dsq.py` — PhotonicMLP_DSQ

3. 量化方法详解

3.1 STE (直通估计器)

STE 是最基础的 QAT 方法。其核心思想是在前向传播中对张量进行伪量化 (四舍五入 + 截断)，但在反向传播中直接将梯度"直通"回原始浮点张量。

伪量化公式：

```
x_q = clamp(round(x / scale), qmin, qmax) * scale
```

实现： `src/quantization/ste.py:FakeQuantizeSTE`

- `forward`：按 `scale` 对输入做量化并反量化，输出保持浮点但已落入离散网格。
- `backward`：直接返回 `grad_output`，忽略量化操作本身的导数。

训练噪声注入： 在 PhotonicMLP_STEQ 中，训练时向量化后的权重注入高斯噪声 `noise_std=0.05 * scale`，以模拟硬件噪声并增强鲁棒性。

scale 计算： 静态地从张量最大绝对值推导：

```
scale = max_abs / (2^(bits-1) - 1)
```

3.2 LSQ+ (可学习步长量化增强版)

LSQ+ 在 STE 基础上引入**可学习的 scale 和 zero-point**，并通过精心设计的梯度缩放因子稳定训练。

核心思想： 将量化步长 `scale` 和零点偏移 `zero_point` 都设为 `nn.Parameter`，在反向传播中同时学习。与固定 `scale` 的 STE 相比，LSQ+ 可以自适应地调整量化网格，使量化误差最小化。

伪量化公式：

```
x_int = clamp(round(x / scale) + zero_point, qmin, qmax)
x_q = (x_int - zero_point) * scale
```

梯度设计： `src/quantization/lsqplus.py:LSQPlusFakeQuantize`

- `grad_x`：在截断区间内保留原始梯度，区间外为 0（类似 STE）。
- `grad_scale`：引入 LSQ 论文的缩放因子 $1 / \sqrt{N * qmax}$ 稳定训练。
- `grad_zero_point`：对 `zero-point` 单独求导，支持非对称量化。

初始化策略： 第一次 forward 时，从数据统计初始化：

- `scale = (mean + 3*std) / qmax`
- `zero_point = -min(x) / scale`（无符号时）或 0（有符号时）

量化参数独立学习率： LSQ+ 的 `scale` 和 `zero-point` 使用基础学习率的 0.1 倍，避免与大权重梯度竞争。

3.3 DSQ（可微软量化）

DSQ 通过 `tanh` 函数平滑量化过程中的舍入操作，避免了 STE 中梯度与真实量化函数不匹配的问题。

核心思想： 不使用硬舍入（`round`），而是将小数部分通过 `tanh` 平滑映射：

```
soft_frac = tanh(temp * (frac - 0.5)) / (2 * tanh(0.5)) + 0.5
x_q = (floor(x/scale) + soft_frac) * scale
```

温度退火： 训练过程中逐步增大 `temp`：

- 早期 `temp` 较小：软量化，梯度连续，训练稳定；
- 后期 `temp` 增大：趋近硬量化，推理时直接使用 `round`。

实现： `src/quantization/dsq.py:DSQFakeQuantize`

- `forward`：根据当前 `temp` 计算软量化输出。
- `backward`：通过 `tanh` 的导数计算真实梯度，而非 STE 的直通。

- 推理时 (not self.training)：自动回退到硬舍入，确保推理一致性。

3.4 三种方法对比

特性	STE	LSQ+	DSQ
scale 来源	静态 (max-abs)	可学习 (nn.Parameter)	可学习 (nn.Parameter)
zero-point	无	可学习	无
梯度方式	直通 (STE)	解析梯度 + LSQ 缩放	tanh 软量化真实梯度
训练噪声	高斯噪声注入	无 (学习过程自动适应)	无 (温度退火替代)
温度退火	无	无	有 (软→硬)
训练 epoch	10	15	15
适用场景	基线对照	高精度要求	梯度敏感任务

4. 训练流程

训练循环统一封装于 `src/training/common.py`：

```
def train_epoch(model, loader, criterion, optimizer, device):
    # 遍历 DataLoader, 前向 + 反向 + 优化
    # 返回 (avg_loss, accuracy)

def evaluate(model, loader, criterion, device):
    # 验证集评估, 返回 (avg_loss, accuracy)
```

数据加载：`src/data/loaders.py`

- `get_mnist_loaders_local()`：从 `data/processed/*.npz` 加载；
- `get_mnist_loaders_torchvision()`：通过 `torchvision` 自动下载。

训练超参数：

方法	Epochs	优化器	学习率	批量大小	损失函数
FP	3	Adam	0.005	256	CrossEntropy
STE	10	Adam	0.005	256	CrossEntropy

方法	Epochs	优化器	学习率	批量大小	损失函数
LSQ+	15	AdamW + CosineAnnealing	0.005 (权重), 0.0005 (量化)	256	CrossEntropy
DSQ	15	Adam	0.005	256	CrossEntropy

训练后导出： 每个训练脚本在验证准确率超过 90% 后，调用 `src/utils/io.py` 中的导出函数将权重保存为 INT4 `.npy` 文件，同时保存量化参数（scale、zero-point）。

5. 部署到光子推理

训练完成后，模型通过以下路径部署到光子硬件：

```

PyTorch 训练模型
|
v
导出 INT4 权重 (w1_int4.npy, w2_int4.npy, w3_int4.npy)
|
v
导出量化参数 (scale, zero-point)
|
v
NumPy 推理引擎 (numpy_steq / numpy_lsplus / numpy_dsq)
|
v
光子模拟器 (8x2 tiling, optical_mac_tiling)
|
v
远程光子硬件 (osimulator + entrance)

```

8x2 Tiling： `src/inference/simulator.py:optical_mac_tiling`

- 输入维度按 8 切分，输出维度按 2 切分；
- 每个 tile 对应一个光子 MAC 阵列；
- 默认使用 `np.matmul` 作为软件回退；当 `use_simulator=True` 时调用实际光子模拟器。

INT4 推理流程（以 STE 为例）：

1. 输入图像 `[0, 255]` 归一化到 `[0, 1]`，再量化为 `uint4 [0, 15]`；
2. 第一层 MAC： `x_uint4 @ w1_int4`，ReLU 后按 `scale_h1` 重新量化为 `uint4`；

3. 第二层 MAC: $h1_uint4 @ w2_int4$, ReLU 后按 $scale_h2$ 重新量化为 $uint4$;
4. 第三层 MAC: $h2_uint4 @ w3_int4$, argmax 得预测标签。

LSQ+ 在此基础上增加了 zero-point 补偿: $(x_int - zp) * (w_int - zp)$, 再通过 $scale$ 乘积还原实值。